

实现一个小型编译程序

本课程设计任务：

实现一个小型编译程序。

输入：高级语言源程序

输出：四元式程序（必做）

汇编语言程序（选做）

小型编译程序执行分两个阶段：第一阶段，将高级语言源程序翻译成四元式程序；第二阶段，将四元式程序翻译成汇编语言目标程序。如图 1 所示：

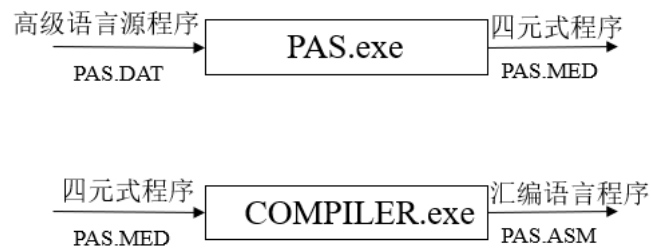


图 1 执行过程示意图

本次课程设计要求所有同学完成小型编译程序的第一阶段（必做），第二阶段为选做题目（完成加分）。

下例为一个名为 PAS.DAT 的高级语言源程序经过 PAS 的编译，产生名为 PAS.MED 的四元式（中间代码）程序；PAS.MED 经过 COMPILER 编译生成 8086/8088 汇编语言目标程序 PAS.ASM。

```
/*  
*****  
/* pas.dat  
*/  
/* 高级语言源程序  
*/  
/*  
*****  
while (a>b) do  
begin
```

```

        if m>=n then a:=a+1
        else
            while k=h do x:=x+2;
            m:=n+x*(m+y)
        end
#
~

/*****/

/* pas.med */

/* 高级语言生成的四元式文件 */

/*****/

100 (j>, a, b, 102)
101 (j, , , 117)
102 (j>=, m, n, 104)
103 (j, , , 107)
104 (+, a, 1, T1)
105 (:=, T1, , a)
106 (j, , , 112)
107 (j=, k, h, 109)
108 (j, , , 112)
109 (+, x, 2, T2)
110 (:=, T2, , x)
111 (j, , , 107)
112 (+, m, y, T3)
113 (*, x, T3, T4)
114 (+, n, T4, T5)
115 (:=, T5, , m)
116 (j, , , 100)
117

/*****/

/* pas.asm */

/* 由四元式文件生成的汇编文件 */

/*****/

```

略

小型编译程序关于高级语言的规定

高级语言程序具有四种基本结构：顺序结构、选择结构、循环结构和过程。

为了便于掌握编译的核心内容，突出重点，简化编译程序的结构，同时又涵盖高级语言程序的基本结构，我们选取赋值语句、if 语句和 while 语句作为前三种结构的代表，略去了过程结构。实际上，上述三种语句已经基本满足了高级语言的程序设计。因此，我们仅考虑由下面产生式所定义的程序语句：

$$S \rightarrow \text{if } B \text{ then } S \text{ else } S \mid \text{while } B \text{ do } S \mid \text{begin } L \text{ end} \mid A$$
$$L \rightarrow S; L \mid S$$
$$A \rightarrow i := E$$
$$B \rightarrow B \wedge B \mid B \vee B \mid \neg B \mid (B) \mid i \text{ rop } i \mid i$$
$$E \rightarrow E + E \mid E * E \mid (E) \mid i$$

其中，各非终结符的含义如下：

S——语句；

L——语句串；

A——赋值句；

B——布尔表达式；

E——算术表达式。

各终结符的含义如下：

i——整型变量或常数，布尔变量或常数；

rop——六种关系运算符的代表;

——起语句分隔符作用;

:= ——赋值符号;

$\neg$ ——逻辑非运算符“not”;

$\wedge$ ——逻辑与运算符“and”;

$\vee$ ——逻辑或运算符“or”;

+ ——算术加运算符;

* ——算术乘运算符;

(——左括号;

) ——右括号。

注意，六种关系运算符分别为

< 小于 <= 小于等于 <> 不等于

> 大于 >= 大于等于 = 等于

关于表达式的运算，我们规定由高到低优先顺序为算术运算、关系运算、布尔运算；并且服从左结合规则。算术运算符优先级的顺序依次为“()”、“*”、“+”；布尔运算符优先级的顺序依次为“ $\neg$ ”、“ $\wedge$ ”、“ $\vee$ ”；六个关系运算符优先级相同。

我们规定的程序是由一条语句或由 begin 和 end 嵌套起来的复合语句组成的，并且规定在语句末要加上“#~”表示程序结束。下面给出的是符合规定的程序示例：

```
/******/  
begin  
  A:=A+B*C;  
  C:=A+2;  
  while A<C and B<D do
```

```
while A>B do
    if M=N then C:=D
    else while A<=D do
        A:=D
End
#~
```

小型编译程序关于单词的内部定义

由于我们规定的程序语句中涉及单词较少, 故在词法分析阶段忽略了单词输入错误的检查, 而将编译程序的重点放在中间代码生成阶段。词法分析器的功能是输入源程序, 输出单词符号。我们规定输出的单词符号格式为如下的二元式:

(单词种别, 单词自身的值)

我们对常量、变量、临时变量、保留关键字(if、while、begin、end、else、then、do 等)、关系运算符、逻辑运算符、分号、括号等, 规定其内部定义如表 1 所示。

表 1 关于单词的内部定义

符 号	种别编码	说 明
sy_if	0	保留字 if
sy_then	1	保留字 then
sy_else	2	保留字 else
sy_while	3	保留字 while
sy_begin	4	保留字 begin
sy_do	5	保留字 do
sy_end	6	保留字 end
a	7	赋值语句
semicolon	8	“;”
e	9	布尔表达式
jinghao	10	“#”
S	11	语句
L	12	复合语句
tempsy	15	临时变量
EA	18	B and(即布尔表达式中 “B ∧ ”)
EO	19	B or(即布尔表达式中 “B ∨ ”)
plus	34	“+”
times	36	“* ”
becomes	38	“:= ”
op_and	39	“and”
op_or	40	“or”
op_not	41	“not”
rop	42	关系运算符
lparent	48	“(”
rparent	49	“) ”
ident	56	变量
intconst	57	整常量

注：单词自身的值对变量而言是指其在变量表中的位置

1. 算术表达式的 LR 分析表

算术表达式文法 G[E]如下：

$$E \rightarrow E + E \mid E * E \mid (E) \mid i$$

分析表由程序生成, 语义程序参考教材设计(可以将文法改写为非二义文法, 并设计语义程序)

2. 布尔表达式的 LR 分析表

布尔表达式的文法如下：

$$B \rightarrow B \wedge B \mid B \vee B \mid \neg B \mid i \text{ rop } i \mid i$$

分析表由程序生成, 语义程序参考教材设计 (可以将文法改写为非二义文法, 并设计语义程序)

3. 程序语句的 LR 分析表

程序语句的文法 $G[S]$ 如下:

$$S \rightarrow \text{if } B \text{ then } S \text{ else } S \mid \text{while } B \text{ do } S \mid \text{begin } L \text{ end} \mid A$$

$$L \rightarrow S; L \mid S$$

分析表由程序生成, 语义程序参考教材设计